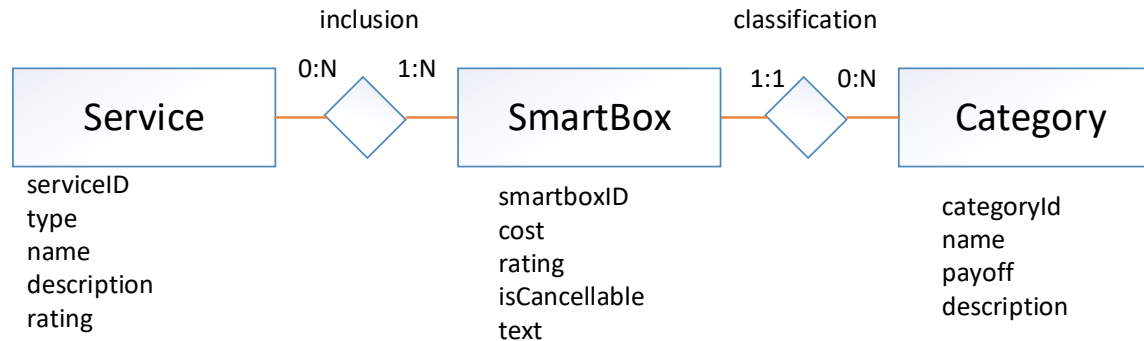


Databases 2

Exam on Jan 23 2023

JPA exercise solutions

Exercise on JPA

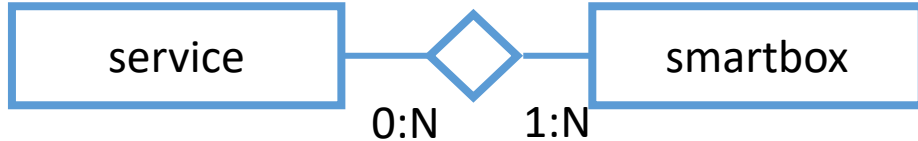


An application of an ecommerce company manages the selling of packaged services called “smart boxes” and allows the inspection of the commercial data. The database is organized according to the above conceptual schema.

The application permits an employee to select a service and check the smart boxes containing it, open one such smart box and check its details including the information of the category it belongs to (e.g., “travel”, “luxury”, “food”, “sports”, etc.). The operator can also access a category and open the list of smart boxes referring to it ordered by cost descending, open one such smart boxes and check the services it contains. The services are in the order of thousands, the smart boxes are in the order of hundreds of thousands and the categories are in the order of tens.

Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.

Relationship inclusion



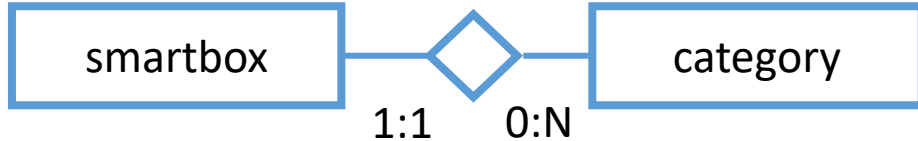
- Service → Smartbox
@ManyToMany

- necessary to show the smart boxes of a service
- Owner = smart box or service
- FetchType can be the default (lazy) (on average 100 smart boxes per service)
- No cascading (smartboxes are related to multiple services)

- Smartbox → Service
@ManyToMany

- necessary to check the details of the services of a selected smart box
- FetchType can be Eager (there are only a few services per smart box)
- No cascading (services are used in multiple smart boxes)

Relationship classification



- Smartbox → Category
@ManyToOne

- necessary to show the category of a smart box
- Owner = smartbox
- FetchType can be the default eager mode
- No cascading useful

- Category → Smartbox
@OneToMany

- necessary to show the smart boxes of a selected category
- FetchType can be lazy (many smart boxes per category)
- Remove can be cascaded
- Order by cost descending

Entity Service

```
@Entity
public class Service{
    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private int serviceId;
    private String type;
    private String name;
    private String description;
    private int rating;

    // lazy can be omitted as it is the default for collections
    @ManyToMany(mappedBy="services", fetch = FetchType.LAZY)
    private List<Smartbox> smartboxes;

    //getters and setters...

}
```

Entity Smartbox

```
@Entity
public class Smartbox{

    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private int smartboxId;
    private int cost;
    private int rating;
    private boolean isCancellable;
    private String text;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "inclusion",
                joinColumns = @JoinColumn(name = "smartboxId"),
                inverseJoinColumn = @JoinColumn(name = "serviceId"))
    private List<Service> services; // owner of the relation

    @ManyToOne(fetch = FetchType.EAGER) // EAGER is default can be omitted
    @JoinColumn(name="category") // owner of the relation
    private Category category;

    //getters and setters...
}
```

Entity Category

```
@Entity
public class Category{
    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private int categoryId;
    private String name;
    private String payoff;
    private String description;

    @OneToMany(mappedBy="category", fetch = FetchType.LAZY,
                cascade=CascadeType.REMOVE)
    @OrderBy("cost DESC")
    private List<Smartbox> smartboxes;

    //getters and setters...
}
```